

main

September 28, 2026

1 Problem 1

```
[1]: import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.pipeline import Pipeline
import matplotlib.pyplot as plt
```

```
[2]: # 1a
rng = np.random.default_rng(seed=598)

n = 1000
p = 20

beta = np.array([2, -1, 0, 0.8, 0, -1.5, 1.2, 0, 0, 0.3, -0.7, 0, 1.8, 0, 0.5,
    ↪0, -1.2, 0, 0.4, 0])
X = rng.standard_normal(size=(n, p))
epsilon = rng.standard_normal(n)
Y = (X @ beta) + epsilon

print(beta.shape)
print(X.shape)
print(epsilon.shape)
print(Y.shape)

# 1b
X_train, X_test, y_train, y_test = train_test_split(
    X,
    Y,
    test_size=0.8,
    random_state=598,
)
```

```
print(X_train.shape)
print(y_train.shape)
print(X_test.shape)
print(y_test.shape)
```

```
(20,)
(1000, 20)
(1000,)
(1000,)
(200, 20)
(200,)
(800, 20)
(800,)
```

```
[3]: # 1c
pipes = {
    'train_mse': [],
    'test_mse': [],
    'pipeline': [],
    'model_size': [],
}
model_sizes = range(1, X_train.shape[1] + 1)

for k in model_sizes:
    pipes['model_size'].append(k)

    if k == X_train.shape[1]:
        best_model_pipeline = Pipeline([
            ('regressor', LinearRegression())
        ])
    else:
        best_model_pipeline = Pipeline([
            ('sfs', SequentialFeatureSelector(
                estimator=LinearRegression(),
                n_features_to_select=k,
                direction='forward'
            )),
            ('regressor', LinearRegression())
        ])

    best_model_pipeline.fit(X_train, y_train)
    pipes['pipeline'].append(best_model_pipeline)

    y_train_pred = best_model_pipeline.predict(X_train)
    train_mse = mean_squared_error(y_train, y_train_pred)
    pipes['train_mse'].append(train_mse)
```

```

y_test_pred = best_model_pipeline.predict(X_test)
test_mse = mean_squared_error(y_test, y_test_pred)
pipes['test_mse'].append(test_mse)

```

```

[4]: # 1d
plt.figure(figsize=(8, 5))
plt.plot(model_sizes, pipes['test_mse'], marker='o', linestyle='-', color='b',
        ↪linewidth=2, label='Test MSE')
plt.plot(model_sizes, pipes['train_mse'], marker='o', linestyle='--',
        ↪color='r', linewidth=2, label='Train MSE')

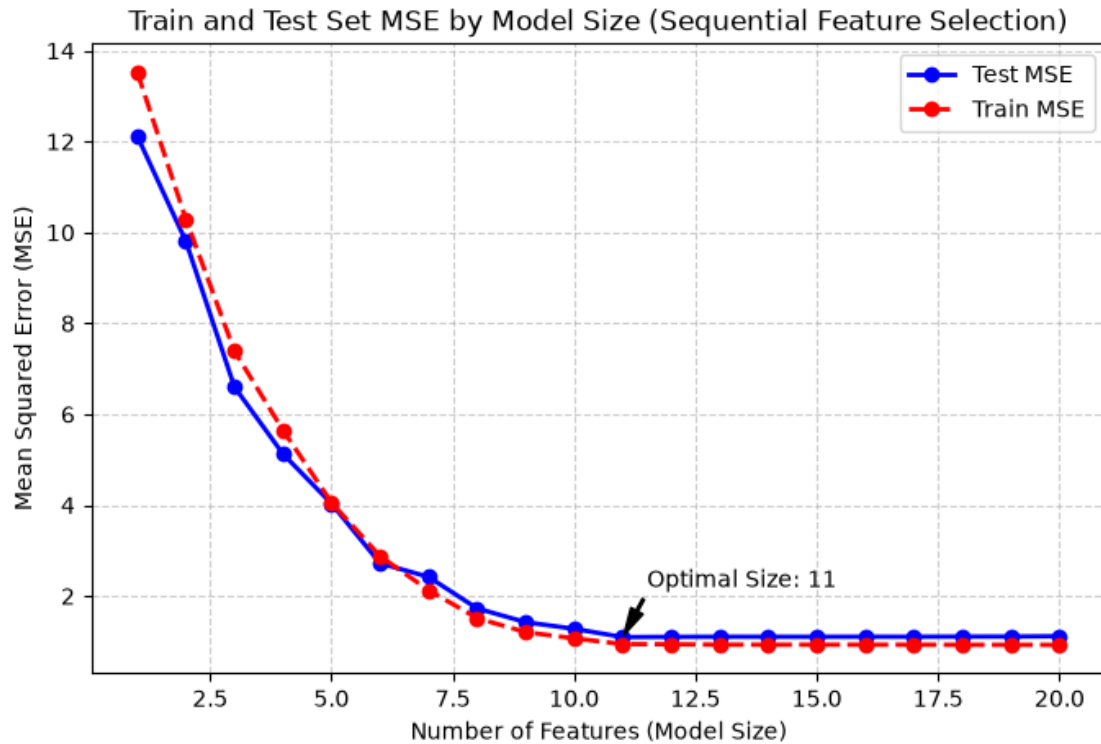
plt.title('Train and Test Set MSE by Model Size (Sequential Feature Selection)')
plt.xlabel('Number of Features (Model Size)')
plt.ylabel('Mean Squared Error (MSE)')
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()

# 1e
# Highlight the optimal model size (lowest test MSE)
best_size = model_sizes[np.argmin(pipes['test_mse'])]
best_mse = min(pipes['test_mse'])
print(f'Model size {best_size} with MSE {best_mse:.4} is the minimum MSE on the
    ↪test set.')
plt.annotate(f'Optimal Size: {best_size}',
            xy=(best_size, best_mse),
            xytext=(best_size + 0.5, best_mse +
    ↪(max(pipes['test_mse'])-min(pipes['test_mse']))*0.1),
            arrowprops=dict(facecolor='black', shrink=0.05, width=1,
    ↪headwidth=6))

plt.show()

```

Model size 11 with MSE 1.095 is the minimum MSE on the test set.



```
[5]: # 1f
best_pipeline: Pipeline = pipes['pipeline'][np.argmin(pipes['test_mse'])]
lr: LinearRegression = best_pipeline.named_steps['regressor']
sfs: SequentialFeatureSelector = best_pipeline.named_steps['sfs']

selected_mask = sfs.get_support()

beta_hat = np.zeros_like(beta, dtype=float)
beta_hat[selected_mask] = lr.coef_

compare = {
    'true': beta,
    'estimated': beta_hat,
    'selected': selected_mask
}
comp_df = pd.DataFrame(compare, index=[f'X_{i}' for i in range(len(beta))])

print(comp_df)
lr
```

	true	estimated	selected
X_0	2.0	2.010178	True
X_1	-1.0	-1.168573	True

X_2	0.0	0.000000	False
X_3	0.8	0.849230	True
X_4	0.0	0.000000	False
X_5	-1.5	-1.466732	True
X_6	1.2	1.234496	True
X_7	0.0	0.000000	False
X_8	0.0	0.000000	False
X_9	0.3	0.370593	True
X_10	-0.7	-0.838337	True
X_11	0.0	0.000000	False
X_12	1.8	1.872613	True
X_13	0.0	0.000000	False
X_14	0.5	0.539317	True
X_15	0.0	0.000000	False
X_16	-1.2	-1.312011	True
X_17	0.0	0.000000	False
X_18	0.4	0.347653	True
X_19	0.0	0.000000	False

[5]: LinearRegression()

It's clear to me now that the best subset selection I performed removed all the zero value coefficients. This makes sense because the dot product of a matrix and a vector looks like this:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix}$$

A zero would impact it like so:

$$y = 0 \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ 0 \end{bmatrix} = \begin{bmatrix} ax \\ cx \end{bmatrix}$$

The coefficient does not add to the model in any way.

```
[6]: # 1g
def coef_distance(beta, beta_hat):
    return np.sqrt(np.sum((beta - beta_hat) ** 2))

pipes['co_est'] = []
for i in range(len(pipes['pipeline'])):

    pipe = pipes['pipeline'][i]
    lr: LinearRegression = pipe.named_steps['regressor']

    if i != X_train.shape[1] - 1:
        sfs: SequentialFeatureSelector = pipe.named_steps['sfs']
        selected_mask = sfs.get_support()
        beta_hat = np.zeros_like(beta, dtype=float)
```

```

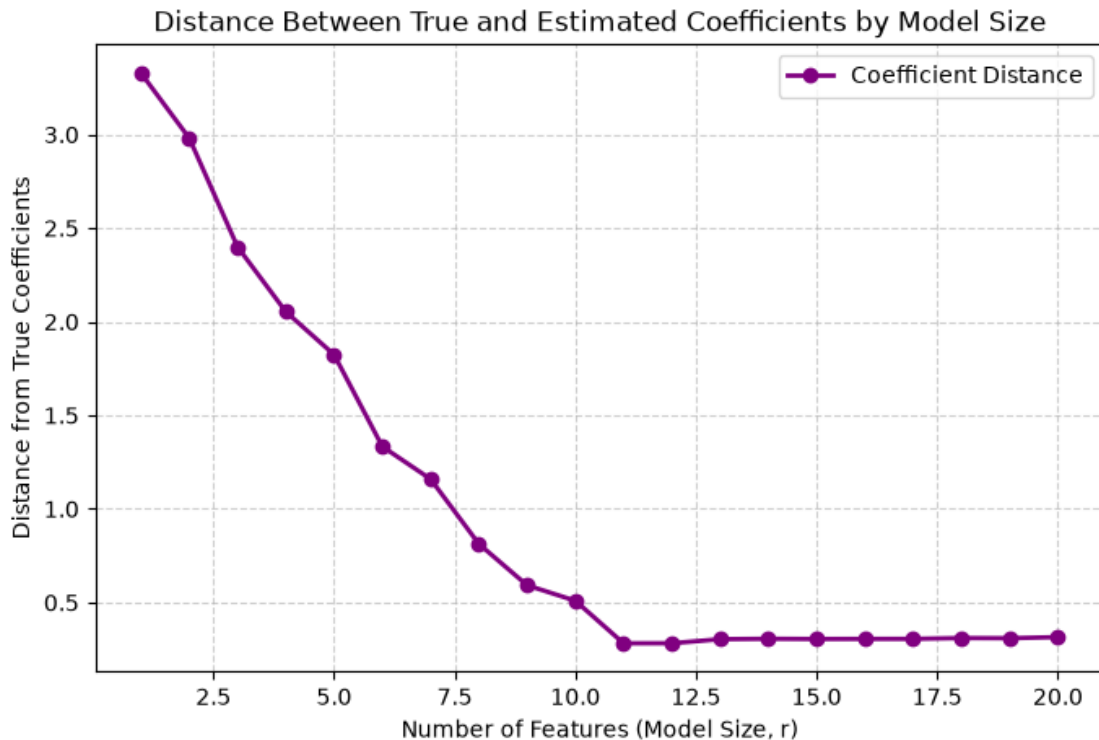
        beta_hat[selected_mask] = lr.coef_
    else:
        beta_hat = lr.coef_

    pipes['co_est'].append(coef_distance(beta, beta_hat))

plt.figure(figsize=(8, 5))
plt.plot(model_sizes, pipes['co_est'], marker='o', linestyle='--',
        color='purple', linewidth=2, label='Coefficient Distance')

plt.title('Distance Between True and Estimated Coefficients by Model Size')
plt.xlabel('Number of Features (Model Size, r)')
plt.ylabel('Distance from True Coefficients')
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()
plt.show()

```



The distance of the modeled coefficients to the true coefficients starts high and steadily decreases as the number of features approaches what I have found to be the best number of features for this model (11). After we reach the best number of features the distance slightly rebounds, but it's minimal. This is because of how the matrix-vector dot product works which I explained earlier. Before 11 features the model is missing information and cannot successfully create an accurate model. After 11 features the model has extra coefficients so the information is captured but the

extra coefficients slightly perturb the distance and results.

The test MSE plot looks very similar to this one. That is because if the coefficients, or unknowns we are solving for are exactly the same as the true coefficients then the predictions will be as close to matching as possible. Therefore a distance measure of the coefficients being approximately zero would also give us a MSE of approximately zero. The magnitudes are different because MSE takes an average and the distance function takes a square root.

2 Problem 2

```
[7]: from ucimlrepo import fetch_ucirepo
import math
```

```
[8]: # fetch dataset
parkinsons_telemonitoring = fetch_ucirepo(id=189)

# data (as pandas dataframes)
X = parkinsons_telemonitoring.data.features
y = parkinsons_telemonitoring.data.targets

# metadata
print(parkinsons_telemonitoring.metadata)

# variable information
print(parkinsons_telemonitoring.variables)

y = y.drop(columns=['motor_UPDRS'])
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=598,
)

y_train = y_train.squeeze()
y_test = y_test.squeeze()

print(X_train.shape)
print(y_train.shape)
print(X_test.shape)
print(y_test.shape)
```

```
{'uci_id': 189, 'name': 'Parkinsons Telemonitoring', 'repository_url':
'https://archive.ics.uci.edu/dataset/189/parkinsons+telemonitoring', 'data_url':
'https://archive.ics.uci.edu/static/public/189/data.csv', 'abstract': "Oxford
Parkinson's Disease Telemonitoring Dataset", 'area': 'Health and Medicine',
'tasks': ['Regression'], 'characteristics': ['Tabular'], 'num_instances': 5875,
'num_features': 19, 'feature_types': ['Integer', 'Real'], 'demographics':
```

['Age', 'Sex'], 'target_col': ['motor_UPDRS', 'total_UPDRS'], 'index_col':
 ['subject#'], 'has_missing_values': 'no', 'missing_values_symbol': None,
 'year_of_dataset_creation': 2009, 'last_updated': 'Fri Nov 03 2023',
 'dataset_doi': '10.24432/C5ZS3N', 'creators': ['Athanasios Tsanas', 'Max
 Little'], 'intro_paper': {'ID': 229, 'type': 'NATIVE', 'title': "Accurate
 Telemonitoring of Parkinson's Disease Progression by Noninvasive Speech Tests",
 'authors': 'A. Tsanas, Max A. Little, P. McSharry, L. Ramig', 'venue': 'IEEE
 Transactions on Biomedical Engineering', 'year': 2010, 'journal': None, 'DOI':
 None, 'URL': 'https://www.semanticscholar.org/paper/1fdf33b6d8b1bdb38866ba824c1d
 caecdfb6bdd6', 'sha': None, 'corpus': None, 'arxiv': None, 'mag': None, 'acl':
 None, 'pmid': '19932995', 'pmcid': None}, 'additional_info': {'summary': "This
 dataset is composed of a range of biomedical voice measurements from 42 people
 with early-stage Parkinson's disease recruited to a six-month trial of a
 telemonitoring device for remote symptom progression monitoring. The recordings
 were automatically captured in the patient's homes.\r\n\r\nColumns in the table
 contain subject number, subject age, subject gender, time interval from baseline
 recruitment date, motor UPDRS, total UPDRS, and 16 biomedical voice measures.
 Each row corresponds to one of 5,875 voice recording from these individuals. The
 main aim of the data is to predict the motor and total UPDRS scores
 ('motor_UPDRS' and 'total_UPDRS') from the 16 voice measures.\r\n\r\nThe data is
 in ASCII CSV format. The rows of the CSV file contain an instance corresponding
 to one voice recording. There are around 200 recordings per patient, the subject
 number of the patient is identified in the first column. For further information
 or to pass on comments, please contact Athanasios Tsanas
 (tsanasthanasis@gmail.com) or Max Little
 (littlem@physics.ox.ac.uk).\r\n\r\nFurther details are contained in the
 following reference -- if you use this dataset, please cite:\r\nAthanasios
 Tsanas, Max A. Little, Patrick E. McSharry, Lorraine O. Ramig
 (2009),\r\n'Accurate telemonitoring of Parkinson's disease progression by non-
 invasive speech tests',\r\nIEEE Transactions on Biomedical Engineering (to
 appear).\r\n\r\nFurther details about the biomedical voice measures can be found
 in:\r\nMax A. Little, Patrick E. McSharry, Eric J. Hunter, Lorraine O. Ramig
 (2009), \r\n'Suitability of dysphonia measurements for telemonitoring of
 Parkinson's disease', \r\nIEEE Transactions on Biomedical Engineering,
 56(4):1015-1022\r\n", 'purpose': None, 'funded_by': None, 'instances_represent':
 None, 'recommended_data_splits': None, 'sensitive_data': None,
 'preprocessing_description': None, 'variable_info': "subject# - Integer that
 uniquely identifies each subject\r\nage - Subject age\r\nsex - Subject gender
 '0' - male, '1' - female\r\nntest_time - Time since recruitment into the trial.
 The integer part is the number of days since recruitment. \r\nmotor_UPDRS -
 Clinician's motor UPDRS score, linearly interpolated\r\ntotal_UPDRS -
 Clinician's total UPDRS score, linearly
 interpolated\r\nJitter(%), Jitter(Abs), Jitter:RAP, Jitter:PPQ5, Jitter:DDP -
 Several measures of variation in fundamental frequency\r\nShimmer, Shimmer(dB), Sh
 immer:APQ3, Shimmer:APQ5, Shimmer:APQ11, Shimmer:DDA - Several measures of
 variation in amplitude\r\nNHR, HNR - Two measures of ratio of noise to tonal
 components in the voice\r\nRPDE - A nonlinear dynamical complexity
 measure\r\nDFA - Signal fractal scaling exponent\r\nPPE - A nonlinear measure of

fundamental frequency variation \r\n", 'citation': None}}

	name	role	type	demographic \
0	subject#	ID	Integer	NaN
1	age	Feature	Integer	Age
2	test_time	Feature	Continuous	NaN
3	Jitter(%)	Feature	Continuous	NaN
4	Jitter(Abs)	Feature	Continuous	NaN
5	Jitter:RAP	Feature	Continuous	NaN
6	Jitter:PPQ5	Feature	Continuous	NaN
7	Jitter:DDP	Feature	Continuous	NaN
8	Shimmer	Feature	Continuous	NaN
9	Shimmer(dB)	Feature	Continuous	NaN
10	Shimmer:APQ3	Feature	Continuous	NaN
11	Shimmer:APQ5	Feature	Continuous	NaN
12	Shimmer:APQ11	Feature	Continuous	NaN
13	Shimmer:DDA	Feature	Continuous	NaN
14	NHR	Feature	Continuous	NaN
15	HNR	Feature	Continuous	NaN
16	RPDE	Feature	Continuous	NaN
17	DFA	Feature	Continuous	NaN
18	PPE	Feature	Continuous	NaN
19	motor_UPDRS	Target	Continuous	NaN
20	total_UPDRS	Target	Continuous	NaN
21	sex	Feature	Binary	Sex

		description	units	missing_values
0	Integer that uniquely identifies each subject	None		no
1	Subject age	None		no
2	Time since recruitment into the trial. The int...	None		no
3	Several measures of variation in fundamental f...	None		no
4	Several measures of variation in fundamental f...	None		no
5	Several measures of variation in fundamental f...	None		no
6	Several measures of variation in fundamental f...	None		no
7	Several measures of variation in fundamental f...	None		no
8	Several measures of variation in amplitude	None		no
9	Several measures of variation in amplitude	None		no
10	Several measures of variation in amplitude	None		no
11	Several measures of variation in amplitude	None		no
12	Several measures of variation in amplitude	None		no
13	Several measures of variation in amplitude	None		no
14	Two measures of ratio of noise to tonal compon...	None		no
15	Two measures of ratio of noise to tonal compon...	None		no
16	A nonlinear dynamical complexity measure	None		no
17	Signal fractal scaling exponent	None		no
18	A nonlinear measure of fundamental frequency v...	None		no
19	Clinician's motor UPDRS score, linearly interp...	None		no
20	Clinician's total UPDRS score, linearly interp...	None		no
21	Subject sex '0' - male, '1' - female	None		no

```
(4112, 19)
(4112,)
(1763, 19)
(1763,)
```

```
[9]: # 2a
def selected_features(pipe):
    sfs = pipe.named_steps["sfs"]
    return ", ".join(X_train.columns[sfs.get_support()])

pipes = {
    'train_mse': [],
    'test_mse': [],
    'pipeline': [],
    'model_size': [],
    'aic': [],
    'bic': [],
    'r2': [],
    'cp': [],
    'coefs': [],
}
N, M = X_train.shape
model_sizes = range(1, 9)

full_model = LinearRegression().fit(X_train, y_train)
y_pred_full = full_model.predict(X_train)
y_avg = np.sum(y_train) / N
tss_full = np.sum((y_train - y_avg) ** 2)
mse_full = mean_squared_error(y_train, y_pred_full)
rss_full = np.sum((y_train - y_pred_full) ** 2)
sigma2_hat = rss_full / (N - M - 1)

for k in model_sizes:
    best_model_pipeline = Pipeline([
        ('sfs', SequentialFeatureSelector(
            estimator=LinearRegression(),
            n_features_to_select=k,
            direction='forward'
        )),
        ('regressor', LinearRegression())
    ])

    best_model_pipeline.fit(X_train, y_train)

    y_train_pred = best_model_pipeline.predict(X_train)
    train_mse = mean_squared_error(y_train, y_train_pred)
    y_test_pred = best_model_pipeline.predict(X_test)
```

```

test_mse = mean_squared_error(y_test, y_test_pred)

log_likelihood = -(N / 2) * (math.log(2 * math.pi) + math.log(train_mse) +
↪1)
p = k + 1
aic = 2 * p - 2 * log_likelihood
bic = math.log(N) * p - 2 * log_likelihood
rss = np.sum((y_train - y_train_pred) ** 2)
r2 = 1 - (rss / (N - p)) / ((tss_full) / (N - 1))
cp = (rss / sigma2_hat) - N + 2 * p

pipes['train_mse'].append(train_mse)
pipes['test_mse'].append(test_mse)
pipes['pipeline'].append(best_model_pipeline)
pipes['model_size'].append(k)
pipes['aic'].append(aic)
pipes['bic'].append(bic)
pipes['r2'].append(r2)
pipes['cp'].append(cp)

print(f'-----\nModel size: {k}, predictors:
↪[{selected_features(best_model_pipeline)}]\ntrain mse: {train_mse}, test mse:
↪{test_mse}, aic: {aic}, bic: {bic}, r2: {r2}, Cp: {cp}')

```

```

-----
Model size: 1, predictors: [age]
train mse: 101.2319847635533, test mse: 108.67337172014066, aic:
30660.159997984734, bic: 30672.803327599006, r2: 0.09454474320660222, Cp:
332.8600966684062
-----
Model size: 2, predictors: [age, HNR]
train mse: 99.17069173639733, test mse: 107.01184520386897, aic:
30577.566957540956, bic: 30596.53195196236, r2: 0.11276581685286025, Cp:
244.43498079995334
-----
Model size: 3, predictors: [age, HNR, DFA]
train mse: 97.3324058295774, test mse: 104.7757613154166, aic:
30502.629216955134, bic: 30527.915876183673, r2: 0.12900013457232207, Cp:
165.79277518744675
-----
Model size: 4, predictors: [age, Shimmer:DDA, HNR, DFA]
train mse: 95.97636865941816, test mse: 103.67306427184785, aic:
30446.937926699673, bic: 30478.546250735348, r2: 0.14092580117045206, Cp:
108.3059304651515
-----
Model size: 5, predictors: [age, Shimmer:DDA, HNR, DFA, sex]
train mse: 94.90695033020253, test mse: 102.00424768329167, aic:
30402.862720102774, bic: 30440.792708945584, r2: 0.15029115729561782, Cp:

```

63.39252462852801

Model size: 6, predictors: [age, test_time, Shimmer:DDA, HNR, DFA, sex]
train mse: 94.40537269884129, test mse: 100.85206067362573, aic:
30383.073414333438, bic: 30427.325067983384, r2: 0.1545759187031156, Cp:
43.38924084725841

Model size: 7, predictors: [age, test_time, Shimmer:DDA, HNR, DFA, PPE, sex]
train mse: 94.19150880089049, test mse: 100.0621979857249, aic:
30375.74761087027, bic: 30426.32092932735, r2: 0.15628559085581895, Cp:
36.00742686083959

Model size: 8, predictors: [age, test_time, Shimmer:DDA, NHR, HNR, DFA, PPE, sex]
train mse: 93.96541962484999, test mse: 99.56328020829027, aic:
30367.86565598194, bic: 30424.760639246153, r2: 0.1581056302204038, Cp:
28.089312516721293

```
[10]: fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(15, 8))
fig.suptitle('Best-Subset Selection Metrics by Model Size', fontsize=16, y=1.02)

# Flatten the 2D axes array for easy iteration
axes = axes.flatten()

# Define the data mapping based on your original colors
plots_config = [
    ('train_mse', 'Train MSE', 'red'),
    ('test_mse', 'Test MSE', 'orange'),
    ('aic', 'AIC', 'yellow'), # Note: yellow can be hard to see on white
    ↪backgrounds
    ('bic', 'BIC', 'green'),
    ('r2', 'Adjusted R-squared', 'blue'),
    ('cp', 'Mallows Cp', 'purple')
]

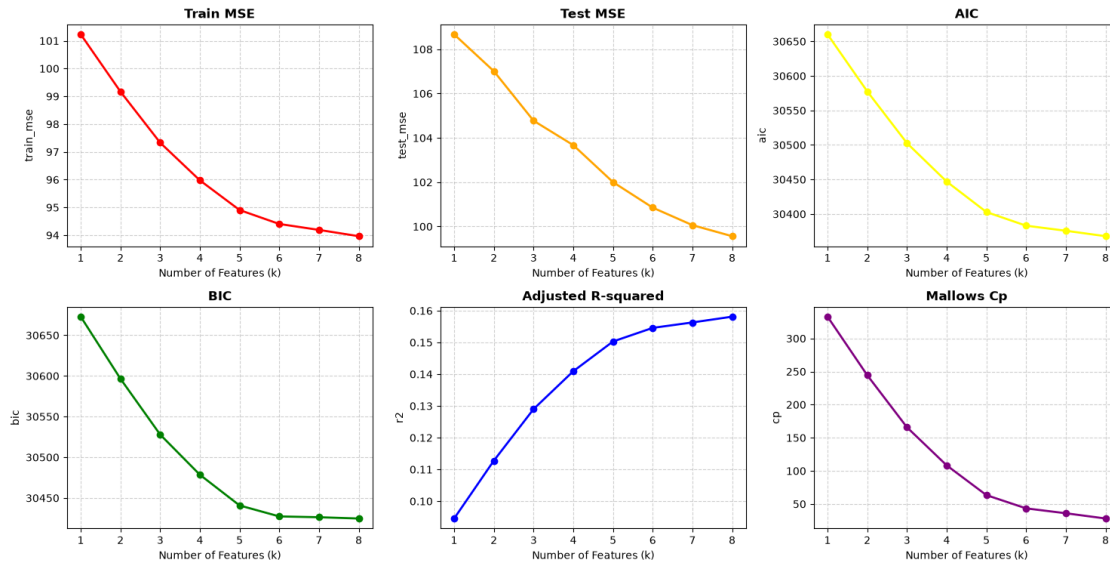
for i, (metric, title, color) in enumerate(plots_config):
    axes[i].plot(model_sizes, pipes[metric], marker='o', linestyle='--', ↪
    ↪color=color, linewidth=2)
    axes[i].set_title(title, fontsize=12, fontweight='bold')
    axes[i].set_xlabel('Number of Features (k)')
    axes[i].set_ylabel(metric)
    axes[i].grid(True, linestyle='--', alpha=0.6)

    # Force integer ticks on the x-axis for model size
    axes[i].set_xticks(model_sizes)

# Automatically adjust spacing between subplots to prevent overlap
```

```
plt.tight_layout()
plt.show()
```

Best-Subset Selection Metrics by Model Size



```
[ ]: # 2b
best_aic = np.argmin(pipes['aic'])
best_bic = np.argmin(pipes['bic'])
best_r2 = np.argmax(pipes['r2'])
best_cp = np.argmin(pipes['cp'])

print(f'Best AIC model: {pipes['model_size'][best_aic]}, train mse: {pipes['train_mse'][best_aic]}, test mse: {pipes['test_mse'][best_aic]}')
print(f'Best BIC model: {pipes['model_size'][best_bic]}, train mse: {pipes['train_mse'][best_bic]}, test mse: {pipes['test_mse'][best_bic]}')
print(f'Best adjusted R2 model: {pipes['model_size'][best_r2]}, train mse: {pipes['train_mse'][best_r2]}, test mse: {pipes['test_mse'][best_r2]}')
print(f'Best Mallows Cp model: {pipes['model_size'][best_cp]}, train mse: {pipes['train_mse'][best_cp]}, test mse: {pipes['test_mse'][best_cp]}')
```

```
Best AIC model: 8, train mse: 93.96541962484999, test mse: 99.56328020829027
Best BIC model: 8, train mse: 93.96541962484999, test mse: 99.56328020829027
Best R2 model: 8, train mse: 93.96541962484999, test mse: 99.56328020829027
Best Mallows Cp model: 8, train mse: 93.96541962484999, test mse:
99.56328020829027
```

```
[12]: # 2c
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

```
rcv = RidgeCV(alphas=np.logspace(-2, 3, 100), store_cv_results=True).
    ↪fit(X_train_scaled, y_train)
rcv.alpha_
```

[12]: 68.92612104349695

```
[13]: # 2d
lcv = LassoCV(max_iter=100000).fit(X_train_scaled, y_train)
mask = lcv.coef_ == 0
print(f'Coefficients identified as zero: {X_train.columns[mask]}')
```

Coefficients identified as zero: Index(['Jitter(%)', 'Shimmer'], dtype='str')

```
[14]: # 2e
X_test_scaled = scaler.transform(X_test)
ridge_preds = rcv.predict(X_test_scaled)
ridge_mse = mean_squared_error(y_test, ridge_preds)
lasso_preds = lcv.predict(X_test_scaled)
lasso_mse = mean_squared_error(y_test, lasso_preds)

print(f"Best subset test MSE: {pipes['test_mse'][7]:.3f}")
print(f"Ridge test MSE: {ridge_mse:.3f}")
print(f"Lasso test MSE: {lasso_mse:.3f}")

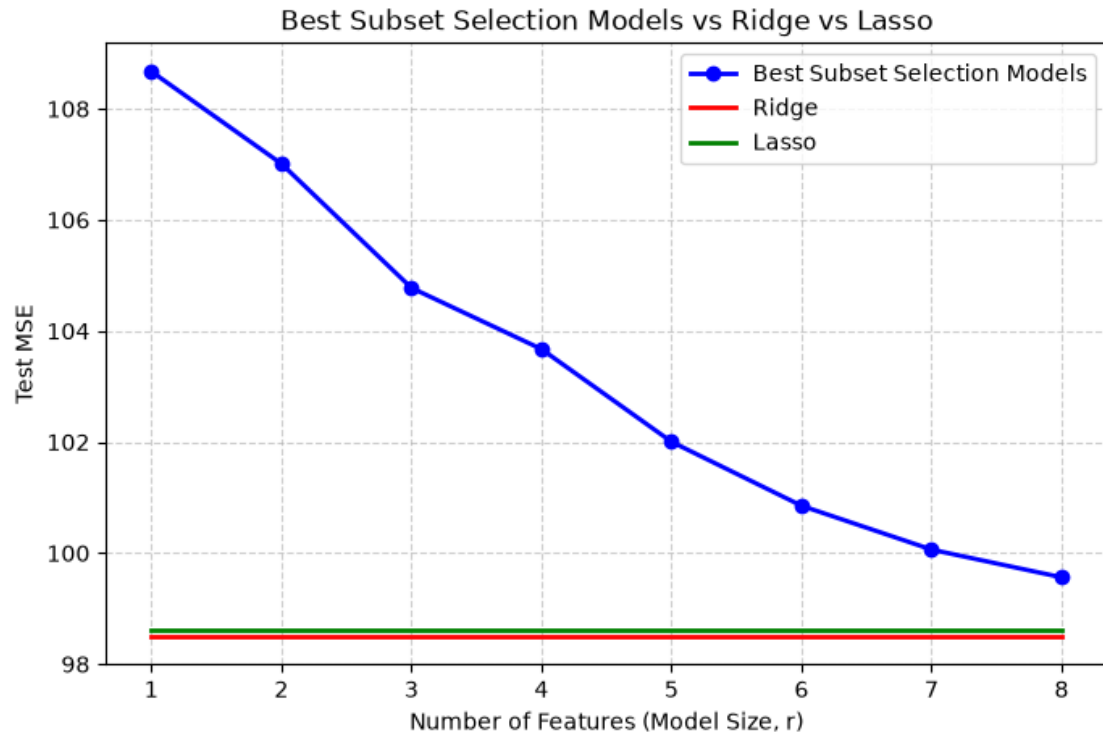
plt.figure(figsize=(8, 5))
plt.plot(model_sizes, pipes['test_mse'], marker='o', linestyle='--', color='b',
    ↪linewidth=2, label='Best Subset Selection Models')
plt.plot(model_sizes, [ridge_mse for i in model_sizes], linestyle='--',
    ↪color='r', linewidth=2, label='Ridge')
plt.plot(model_sizes, [lasso_mse for i in model_sizes], linestyle='--',
    ↪color='g', linewidth=2, label='Lasso')

plt.title('Best Subset Selection Models vs Ridge vs Lasso')
plt.xlabel('Number of Features (Model Size, r)')
plt.ylabel('Test MSE')
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()
plt.show()
```

Best subset test MSE: 99.563

Ridge test MSE: 98.499

Lasso test MSE: 98.620



Ridge is preferred because it has the lowest test MSE (98.499). Lasso performs nearly as well and produces a simpler model by setting two coefficients to zero, so it may be preferable if interpretability is more important than the small predictive improvement from Ridge.